

操作系统并发专题：信号量与 PV 操作方法论

从执行交错到 408 经典同步题的可执行建模方法

408 专题手册系列 · OS 并发模块

定位：PV 不是口诀，而是通用 OS 推理引擎在并发同步下的落地特例

信号量/PV 绝不是背诵伪代码模板，而是通用操作系统推理引擎（OS System Reasoning Engine）在“多线程指令交错”场景下的具体应用：

State (共享状态) → Interleaving (交错转移) → Failure (失效/坏状态) → Property (Safety / Liveness) → Minimization

面对陌生并发题，严格回答解题三问：

1. 第一问（坏状态）：如果完全不加控制，会出现什么错误交错？（寻找 Counterexample，如 Lost Update / 缓冲区溢出）。
2. 第二问（性质与不变量）：我真正要保护的性质是什么？
 - Safety（安全性）：坏事绝不发生（如互斥临界区、 $0 \leq count \leq N$ ）。注意区分宏观逻辑不变量（如动作完成后 $empty + full + in_transition = N$ ）与指令中途的快照；
 - Liveness（活性）：好事最终必发生（如无死锁 No Deadlock、无饥饿 No Starvation）。
3. 第三问（最小机制）：恢复该性质所需的最小约束与 PV 机制是什么？（只禁止危险执行，不防范过度）。

在本手册中，我们严格区分三个核心视级：

- 许可计数（Permit Count）：逻辑世界中的可用资源额度（如 $S = 1$ 临界区名额， $S = N$ 空位额度）。
- 约束映射（Constraint Mapping）：建模世界中将性质推演翻译为 P/V 逻辑顺序过程。
- PV 伪代码（PV Pseudocode）：实现世界中答题卡上最终写出的 P(S) / V(S) 代码。

内容边界

这里定义信号量的许可语义、PV 建模流程、经典同步模式和正确性检查。进程状态转换由进程与调度专题定义；本专题只使用 Running→Blocked、Blocked→Ready 等状态结果。

目录

1 第零章：为什么会有并发问题	5
1.1 并发真正研究的对象是“执行轨迹集合”	5
1.2 综合题：一条 counter++ 的一生	5
1.3 同步不等于互斥	6

1.4	互斥的底座机制：关中断 vs 硬件原子指令	6
2	信号量到底是什么：把并发约束映射为许可计数	7
2.1	最稳定的理解：Permit Counter + Waiting Queue	7
2.2	P/V 的两种常见教材记法	7
2.3	P/V 为什么必须是原子操作	8
2.4	二元信号量与 Mutex：考试等效，语义并非完全相同	8
3	PV 大题统一建模算法：先建模，再填 P/V	8
3.1	旧式做法的问题	8
3.2	七步法	9
3.3	“先同步 P，再互斥 P”不是宇宙定律	9
3.4	V 的顺序也不要绝对化	10
4	模式一：生产者-消费者——容量计数 + 互斥更新	10
4.1	问题模型与不变量	10
4.2	信号量设计	10
4.3	标准模板	10
4.4	为什么生产和消费要放在锁外	11
4.5	经典死锁轨迹	11
4.6	典型变式	11
5	模式二：读者-写者——群体准入 + 元数据保护	12
5.1	约束结构	12
5.2	读者优先模板	12
5.3	为什么 <code>readcount</code> 本身必须保护	13
5.4	公平性是独立策略维度	13
6	模式三：理发师——有限准入 + 双向会合	14
6.1	模型抽象	14
6.2	经典信号量	14
6.3	它为什么比生产者-消费者多一层	15
7	模式四：哲学家进餐——多资源获取 + 循环等待	15
7.1	真正训练的是资源依赖图	15
7.2	常见死锁规避思路	15
7.3	状态数组 + 私有信号量的阻塞范式	15

8 模式五：吸烟者——条件分派与精确通知	16
8.1 核心不是“材料”，而是多路条件	16
8.2 简洁 408 模板	16
9 模式六：前驱图——把 Happens-Before 边编码成事件信号量	17
9.1 图模型	17
9.2 2022 风格示例	17
9.3 新增模式：Barrier 与 fork/join	18
10 从 PV 到管程：wait 为什么必须“释放并睡眠”	19
10.1 问题不是“wait 会不会释放锁”，而是不能丢失唤醒	19
10.2 标准思维模板	19
10.3 管程是什么	20
11 死锁：从等待依赖到资源状态安全性	20
11.1 死锁到底是什么	20
11.2 等待图与资源分配图	21
11.3 Coffman 四个必要条件	21
11.4 死锁处理的四种策略层级	22
11.5 Prevention：破坏必要条件的机制与代价	22
11.6 State Classification：Safe / Unsafe / Deadlocked 集合关系	22
11.7 Avoidance 与银行家算法 (Banker's Algorithm)	23
11.8 安全算法 (Safety Algorithm) 极简理解	24
11.9 Avoidance vs Detection：考场核心辨析	24
11.10 Deadlock Detection（死锁检测算法）	24
11.11 Recovery：死锁恢复与代价评估	25
11.12 考场判题协议 (Exam Protocol)	26
12 经典 PV 题不要背答案：把它们看成“并发结构模式库”	26
13 408 大题作答模板：从自然语言到 PV 伪代码	26
13.1 第一遍：只翻译业务，不写 P/V	26
13.2 第二遍：建立状态表	27
13.3 第三遍：在“等待点”和“制造条件点”成对放置 P/V	27
13.4 第四遍：四类证明	27
14 高频错因诊断表	27

15 考场速查：PV 十问	28
16 专题总结：把 PV 从代码模板提升为状态约束与语义建模	28
边界检查：四句不能绝对化	29

1 第零章：为什么会有并发问题

1.1 并发真正研究的对象是“执行轨迹集合”

在并发编程中，无法只看静态代码，必须分析动态运行时的指令执行序列：

- 多线程指令交错（Interleaving）：并发运行时，CPU 调度器可以在任意时刻切换线程。线程 T_1 和线程 T_2 的指令在时间轴上交叉拼合，称为“指令交错”。
- 执行轨迹（Execution Trajectory / Trace，记作 σ ）：把一次运行中真实发生的全局指令顺序记录下来，构成的具体流水账叫“执行轨迹”。
- 执行轨迹集合（Set of Trajectories，记作 Σ ）：由于 CPU 调度的随机性，所有可能产生的交错流水账构成集合 $\Sigma = \{\sigma_1, \sigma_2, \dots\}$ 。
- 并发控制的目标：单次运行成功仅代表集合 Σ 中的某一条轨迹 σ 正确。并发控制的本质，是通过 P/V 操作修剪这个轨迹集合 Σ ，剔除所有引发数据竞争和死锁的错误轨迹。

设有两个执行流 T_1, T_2 共同操作共享状态 S 。某一条具体交错轨迹表示为：

$$\sigma = T_1^1, T_2^1, T_1^2, T_2^2, \dots$$

并发程序正确性定义为：对所有允许交错轨迹，系统状态都必须满足安全不变量 I ：

$$\forall \sigma \in \text{Allowed Interleavings}, \quad I(S_\sigma) = \text{true}$$

符号 / 术语	严谨定义与直观含义解析
σ (Sigma)	某一条具体的执行轨迹。一次运行中 CPU 按时间顺序执行的指令步骤流水账。
T_i^j	进程/线程 T_i 的第 j 个原子步骤。例如 T_1^1 表示线程 1 执行的第一条汇编指令。
S_σ	沿着轨迹 σ 执行完毕后的共享状态。如变量 <code>counter</code> 的最终值。
$I(S)$	系统不变量（Invariant）。无论何时都绝对不能被破坏的安全业务规则（如 $0 \leq count \leq N$ ）。
$\forall \sigma \in \text{Allowed Interleavings}$	对允许轨迹集合中的任意一条轨迹。要求对所有合法交错组合全部成立。

并发篇第一原则

并发不是“执行顺序不确定”这么简单，而是：程序员不能假定调度器会在方便的位置切换；高层语句又可能由多条底层步骤组成。因此正确性必须对所有允许交错成立。

1.2 综合题：一条 `counter++` 的一生

程序员看到：

```
counter++
```

机器可能执行：

```
load counter
```

```
add 1
store counter
```

若初值为 5，两个线程可能发生：

```
T1: load 5
T2: load 5
T1: add  -> 6
T2: add  -> 6
T1: store 6
T2: store 6
```

最终得到 6 而不是 7，称为**丢失更新 (Lost Update)**。

这条最小反例依次推出：

Shared State → Interleaving → Race → Atomicity → Critical Section → Mutual Exclusion

1.3 同步不等于互斥

并发控制中最基本的两个关系必须分开：

关系类型	本质驱动问题	典型工具与直观表现
互斥 (Mutual Exclusion)	哪些关键指令/状态更新不能彼此交错？	Mutex、二元信号量 (Initial=1)、临界区
条件/顺序同步 (Synchronization)	什么条件满足后才能继续？谁必须先于谁？	计数信号量 (Initial=N/0)、事件信号量、条件变量

考试识别

看到“共享变量、共享缓冲区、文件写入、计数器”等，先找**不允许交错的状态更新**；看到“有产品才能消费、A 完成后 B 才能做、资源数量有限”等，先找**等待条件/前驱关系**。很多 PV 大题是两者叠加。

1.4 互斥的底座机制：关中断 vs 硬件原子指令

为了实现互斥 (Critical Section)，硬件与系统提供了从单核到多核的底座机制演进：

- 1. **关中断 (Disable Interrupt)**：单 CPU 运行环境下，CPU 通过关闭本地中断 (cli)，防止时钟中断触发调度切换，确保临界区代码不被同一 CPU 上的其他线程抢占。
- 2. **硬件原子指令 (TestAndSet / CAS / LL-SC)**：现代多核 (SMP) 架构下，多 CPU 共享主存。关闭本地 CPU 的中断 ** 无法阻止其他 CPU 核心访问共享内存 **！必须依赖 CPU 硬件层面的原子读改写 (RMW) 指令与缓存一致性/总线锁。

必须确立核心判定边界：

Disable Local Interrupt $\nrightarrow$ Multiprocessor Global Mutual Exclusion

机制 / 方法	核心原理与适用场景	缺陷与代价
Disable Interrupt	关本地 CPU 中断，防调度抢占；适用于单核 CPU 极短内核操作	多核 (SMP) 无效；滥用关中断可能导致系统响应丧失与实时性破坏
Software Peterson	纯软件标志位 (flag + turn) 实现两线程互斥	存在 Busy-waiting (忙等)；依赖内存顺序，难以推广到 N 线程
TestAndSet (TAS)	硬件指令原子性地“读旧值并写新值 1”	存在 Busy-waiting 消耗 CPU 算力 (Spin-lock 自旋锁特征)
Compare-And-Swap	比较当前值是否等于期望值，相等则原子更新	存在 ABA 问题 (需版本号解决)；自旋开销受高并发争用影响

2 信号量到底是什么：把并发约束映射为许可计数

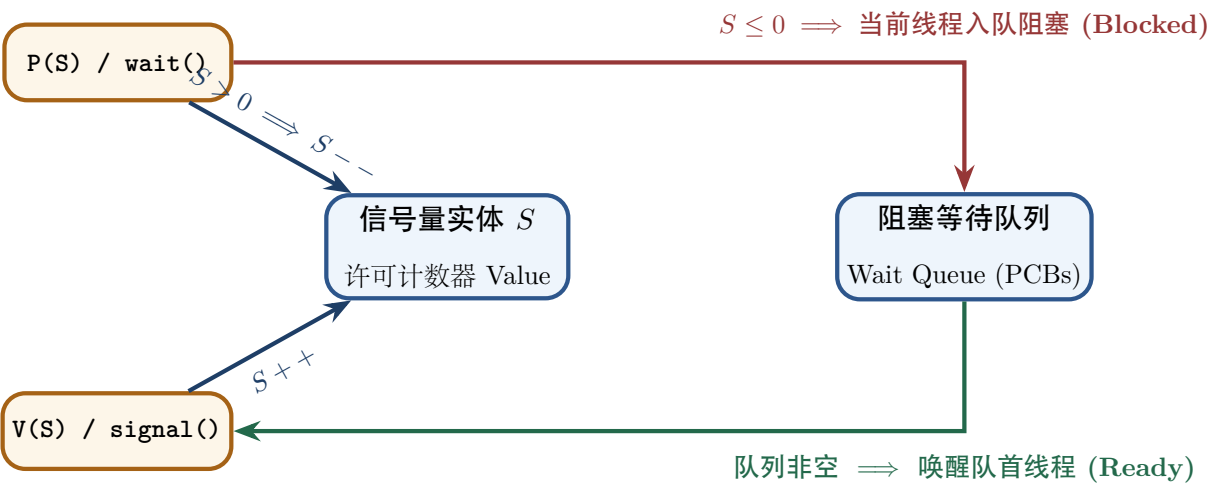
2.1 最稳定的理解：Permit Counter + Waiting Queue

信号量可以理解为：

Semaphore = Permit Counter + Atomic Wait/Wake Mechanism

它不是“神秘整数”，而是把某种现实业务规则映射为具体可消费的许可数量 (Permit Count)：

- $S = 1$ ：当前有 1 个进入资格，可用于单资源互斥；
- $S = N$ ：当前有 N 个同类资源或容量许可；
- $S = 0$ ：某个必须等待的事件尚未发生，或当前没有可消费资源。



初值不是背出来的

问：系统初始时到底已经拥有多少份可以立即被消费的事实/资源/空位/许可？这个数量就是计数信号量初值的来源。

2.2 P/V 的两种常见教材记法

不同教材会采用不同内部表示。做题时首先服从题目定义。

记法 A：可出现负值的经典教学模型

$P(S): S \leftarrow S - 1; \quad S < 0 \Rightarrow \text{当前进程阻塞入队},$
 $V(S): S \leftarrow S + 1; \quad S \leq 0 \Rightarrow \text{从等待队列唤醒一个进程}.$

在这种模型中，负值的绝对值常可解释为等待者数量。

记法 B：计数保持非负的抽象模型

- **wait/P**: 若有许可则原子地消费一个；若无许可则阻塞；
- **post/V**: 增加一个许可；若有等待者则允许其中一个继续竞争运行。

两者语义一致：**P 可能阻塞，V 可能唤醒，且 P/V 自身必须原子。**

2.3 P/V 为什么必须是原子操作

如果 $P(S)$ 自己可以被拆成“读 S 、判断、减一、入队”等可交错步骤，那么两个执行流可能同时认为自己拿到了同一份许可。信号量的正确性因此必须最终依赖更底层的原子机制，如 `test-and-set`、`compare-and-swap`、`fetch-and-add` 或内核的受保护临界区。

抽象层次

硬件原子能力 $\rightarrow$ 锁/阻塞原语 $\rightarrow$ 信号量/条件变量 $\rightarrow$ 并发算法

408 通常从信号量层开始做题，但理解下面一层能解释“为什么 P/V 本身可以被信任”。

2.4 二元信号量与 Mutex：考试等效，语义并非完全相同

408 里常说“二元信号量可实现互斥”，这是正确的考试模型。但严格设计上：

- **mutex 强调所有权 (ownership)**：谁加锁，通常由谁解锁；
- **semaphore 强调许可数量**：一个执行流 P ，另一个执行流 V 完全可以有合理语义，例如事件同步。

因此应记：

Binary Semaphore can implement mutual exclusion, but is not conceptually identical to a mutex.

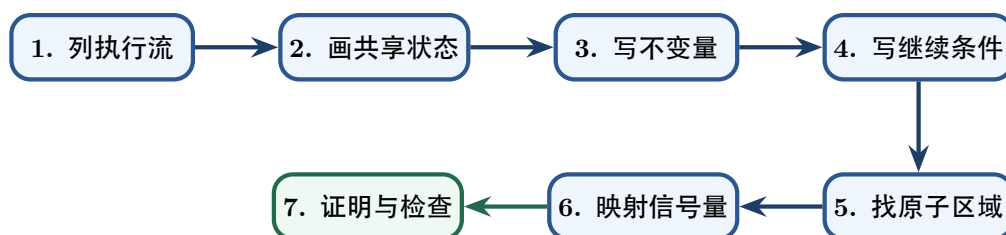
3 PV 大题统一建模算法：先建模，再填 P/V**3.1 旧式做法的问题**

“找互斥、找同步、塞 P/V”很实用，但遇到复杂变式时容易机械。更稳定的方法是从共享状态和不变量出发。

3.2 七步法

PV 通用七步法

1. 列执行流：有哪几类进程/线程？每类反复执行什么业务动作？
2. 画共享状态：共享缓冲区、计数器、资源、标志位、队列分别是什么？
3. 写不变量：哪些关系永远不能被破坏？如 $0 \leq count \leq N$ 。
4. 写继续条件：每类执行流在什么谓词成立时才能继续？
5. 找原子区域：哪些多步修改必须表现得像一个整体？
6. 把约束映射成信号量：互斥许可、资源数量、事件前驱分别需要什么信号量及初值？
7. 证明与检查：Safety、Deadlock、Liveness、Starvation、锁粒度和 P/V 完备性。



3.3 “先同步 P，再互斥 P” 不是宇宙定律

在标准有界缓冲区中：

```

P(empty);
P(mutex);
... put item ...
V(mutex);
V(full);
  
```

是正确经典模板。

错误写法：

```

P(mutex);
P(empty);  // 若 empty==0, 则持有 mutex 睡眠
  
```

可能死锁，因为只有消费者进入同一临界区才能制造新的 empty，但 mutex 已被生产者占有。

真正应记的原则

不要机械背“所有题都先同步后互斥”。真正规律是：

不要在持有某个资源/锁时，等待一个只能由需要该资源/锁的其他执行流才能制造的条件。

复杂题应画等待依赖，而不是套固定顺序。

3.4 V 的顺序也不要绝对化

标准生产者-消费者通常写：

$$V(mutex) \rightarrow V(full)$$

这样先释放临界区，再通知消费者，避免刚唤醒的消费者马上又阻塞在 `mutex` 上。某些简单模型里交换两个 `V` 不一定破坏 `Safety`，但会改变调度/性能，复杂协议里还可能影响语义。因此不要记成“`V` 顺序永远无所谓”。

4 模式一：生产者-消费者——容量计数 + 互斥更新

4.1 问题模型与不变量

容量为 N 的有界缓冲区：

$$0 \leq count \leq N$$

生产者继续条件：

$$count < N$$

消费者继续条件：

$$count > 0$$

同时，插入/删除和缓冲区元数据更新不能被多个执行流破坏。

4.2 信号量设计

信号量对象	初值 (Initial)	现实含义与许可类型
<code>empty</code>	N	可立即消费的空位许可数量（同步信号量）
<code>full</code>	0	可立即消费的产品许可数量（同步信号量）
<code>mutex</code>	1	进入缓冲区结构更新临界区的唯一许可（互斥信号量）

关键守恒关系可辅助检查：

$$empty + full = N$$

（在经典抽象模型、忽略正在 `P/V` 过渡的瞬间进行逻辑理解。）

4.3 标准模板

```
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

producer() {
    while (true) {
        produce_item();
        P(empty);
        P(mutex);
        put_item();
    }
}
```

```
        V(mutex);
        V(full);
    }
}

consumer() {
    while (true) {
        P(full);
        P(mutex);
        get_item();
        V(mutex);
        V(empty);
        consume_item();
    }
}
```

4.4 为什么生产和消费要放在锁外

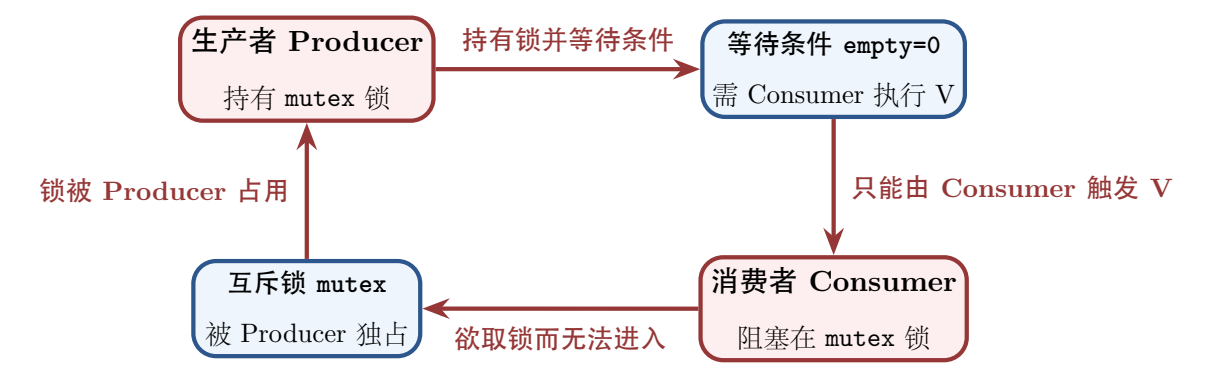
produce_item() 与 consume_item() 若只使用私有数据，不应放入缓冲区 mutex 临界区。否则虽然 Safety 仍可正确，却会把长时间工作串行化，降低并发度。

4.5 经典死锁轨迹

缓冲区已满，若生产者先：

```
P(mutex);    // 成功
P(empty);    // empty=0 -> Blocked
```

则生产者睡眠时仍持有 mutex。消费者即使 Ready/Running，也会阻塞在 P(mutex)，无法取出产品并执行 V(empty)。形成：



本质是等待环。

4.6 典型变式

- 单/多生产者、单/多消费者：基本模板不变；

- 消费一个 A 和一个 B：分别建立 `fullA`、`fullB`，消费者要取得两类许可；
- 多类资源同时获取：开始接近哲学家/多资源分配问题，需要检查循环等待和持有后等待；
- 批量生产/消费 k 个：不能只机械连续执行 k 次 P；要明确题目是否要求“整批原子获得”，以及容量/公平性要求。

5 模式二：读者-写者——群体准入 + 元数据保护

5.1 约束结构

约束：

- 读-读允许；
- 读-写互斥；
- 写-写互斥。

它不是普通“一把锁保护所有人”，而是**群体准入 (group admission)**：多个读者构成一个共享访问群体，第一个读者负责关闭写者入口，最后一个读者负责重新开放。

5.2 读者优先模板

```
semaphore rw_mutex = 1; // 数据区读写锁
semaphore r_mutex  = 1; // 保护 readcount
int readcount = 0;

reader() {
    P(r_mutex);
    if (readcount == 0)
        P(rw_mutex);    // 第一个读者封锁写者
    readcount++;
    V(r_mutex);

    read_data();

    P(r_mutex);
    readcount--;
    if (readcount == 0)
        V(rw_mutex);    // 最后一个读者释放写者
    V(r_mutex);
}

writer() {
    P(rw_mutex);
    write_data();
    V(rw_mutex);
}
```

5.3 为什么 readcount 本身必须保护

readcount++、判断“是否第一个读者”、readcount--、判断“是否最后一个读者”共同决定群体状态。如果它们交错，就可能让多个读者都误以为自己是第一/最后一个，破坏准入协议。

读者-写者真正训练的东西

它训练的不是“多加一把锁”，而是：**共享数据和管理共享数据的元数据都可能是临界状态**。元数据（如计数器、索引、状态位）常常比业务数据本身更容易被忽略。

5.4 公平性是独立策略维度

上述方案是读者优先：只要读者不断到来，写者可能长期无法获得 rw_mutex，即**写者饥饿**。

可以再增加“入口闸门”和写者计数实现写者优先：

```
int read_count = 0;
int write_count = 0;
semaphore r_mutex = 1;
semaphore w_mutex = 1;
semaphore rw_mutex = 1;
semaphore read_try = 1;

writer() {
    P(w_mutex);
    write_count++;
    if (write_count == 1) P(read_try);
    V(w_mutex);

    P(rw_mutex);
    write_database();
    V(rw_mutex);

    P(w_mutex);
    write_count--;
    if (write_count == 0) V(read_try);
    V(w_mutex);
}

reader() {
    P(read_try);
    P(r_mutex);
    read_count++;
    if (read_count == 1) P(rw_mutex);
    V(r_mutex);
    V(read_try);

    read_database();
}
```

```
P(r_mutex);
read_count--;
if (read_count == 0) V(rw_mutex);
V(r_mutex);
}
```

考试层次

408 常见题主要要求读者优先、写者优先或给定策略下填 PV。解题时先确定题目要求的公平策略，再决定是否需要额外闸门；不要把某一版模板当成唯一“读者-写者答案”。

6 模式三：理发师——有限准入 + 双向会合

6.1 模型抽象

理发店有 M 个等待座位。顾客到达：有空位则等待，无空位则离开；理发师无顾客时睡眠。

这个题可以理解为：

- 顾客产生服务请求；
- 理发师消费服务请求；
- 等待椅是有限容量；
- 顾客与理发师还需完成一次 rendezvous（会合）。

6.2 经典信号量

信号量 / 共享变量	初值 (Initial)	含义与作用
customers	0	正在等待服务的顾客数量（同步信号量）
barbers	0	已准备好接待顾客的理发师许可（同步信号量 / 双向会合）
mutex	1	保护 waiting_customers 计数器的互斥信号量
waiting_customers	0	当前在等待椅子上坐着的顾客人数（共享整型变量）

```
barber() {
    while (true) {
        P(customers);
        P(mutex);
        waiting_customers--;
        V(barbers);
        V(mutex);
        cut_hair();
    }
}
```

```
customer() {
    P(mutex);
    if (waiting_customers < CHAIRS) {
        waiting_customers++;
        V(customers);
        V(mutex);
        P(barbers);
        get_haircut();
    } else {
        V(mutex);
        leave_shop();
    }
}
```

6.3 它为什么比生产者-消费者多一层

生产者-消费者只需确保“产品存在”；理发师问题还要让一个具体顾客知道“已有理发师可以服务”，因此多了 `barbers` 这一会合信号。题目若增加多名理发师、不同服务等级或排队规则，就会进一步引入身份匹配与公平性。

7 模式四：哲学家进餐——多资源获取 + 循环等待

7.1 真正训练的是资源依赖图

最朴素方案：每人先左后右。若所有哲学家都拿到左筷子：

$$P_0 \rightarrow C_1 \rightarrow P_1 \rightarrow C_2 \rightarrow \cdots \rightarrow P_{N-1} \rightarrow C_0 \rightarrow P_0$$

形成循环等待。

7.2 常见死锁规避思路

1. 限制准入：最多允许 $N - 1$ 人同时尝试拿筷子；
2. 资源全序：所有人按统一编号顺序获取资源，破坏循环等待；
3. 原子化资格判定：在受保护状态中判断左右邻居，只有同时满足才进入 EATING；否则阻塞等待通知。

7.3 状态数组 + 私有信号量的阻塞范式

```
semaphore mutex = 1;
semaphore self[N] = {0};
state[N] = {THINKING};

test(i) {
    left = (i + N - 1) % N;
```

```

    right = (i + 1) % N;
    if (state[i] == HUNGRY &&
        state[left] != EATING &&
        state[right] != EATING) {
        state[i] = EATING;
        V(self[i]);
    }
}

take_forks(i) {
    P(mutex);
    state[i] = HUNGRY;
    test(i);
    V(mutex);
    P(self[i]);
}

put_forks(i) {
    P(mutex);
    state[i] = THINKING;
    test(left(i));
    test(right(i));
    V(mutex);
}

```

这里 `mutex` 保护的是状态判定过程，`self[i]` 表达“哲学家 i 何时获得进餐资格”。

不要过度声称

这种状态判定方案可以设计为**无死锁**，但“绝对无饥饿”还依赖唤醒/调度公平性和具体判定策略。考试若只问死锁避免，不必额外假设强公平性。

8 模式五：吸烟者——条件分派与精确通知

8.1 核心不是“材料”，而是多路条件

供应者每轮制造不同条件，只有匹配的一个吸烟者应继续：

$$Condition_0 \rightarrow Smoker_0, \quad Condition_1 \rightarrow Smoker_1, \quad Condition_2 \rightarrow Smoker_2.$$

最直接的 PV 思路是：为每种可区分事件设置独立同步信号量。

8.2 简洁 408 模板

设 `agent=1` 表示桌面空、允许供应下一组；`offer[3]=0` 表示三种匹配事件尚未发生。

```

provider() {

```



```

    while (true) {
        P(agent);
        choose k in {0,1,2};
        put_materials_for(k);
        V(offer[k]);
    }
}

smoker(k) {
    while (true) {
        P(offer[k]);
        take_materials();
        make_and_smoke();
        V(agent);
    }
}

```

这体现：

复杂条件同步 → 把不同条件编码为不同等待队列/信号量

关于“代理人模式”

增加 Agent/检查者可以把匹配分发移入中间层，但不是 408 必需模板。若多个代理共同消费同一检查信号，必须防止错误消费事件或形成无效唤醒循环；考试中优先为每类条件设置独立信号量。

9 模式六：前驱图——把 Happens-Before 边编码成事件信号量

9.1 图模型

若有有向边：

$$A \rightarrow C$$

表示 A 完成是 C 开始的前置事实。若两操作位于不同执行流，就需要显式同步；若在同一线程中且程序顺序已保证，则通常无需额外信号量。

一条跨线程前驱边

为跨线程边 $A \rightarrow C$ 定义初值为 0 的信号量 S_{AC} ：

$$A; V(S_{AC}) \qquad P(S_{AC}); C$$

初值为 0 正是因为“ A 已完成”这一事实在系统初始时尚不存在。

9.2 2022 风格示例

依赖：

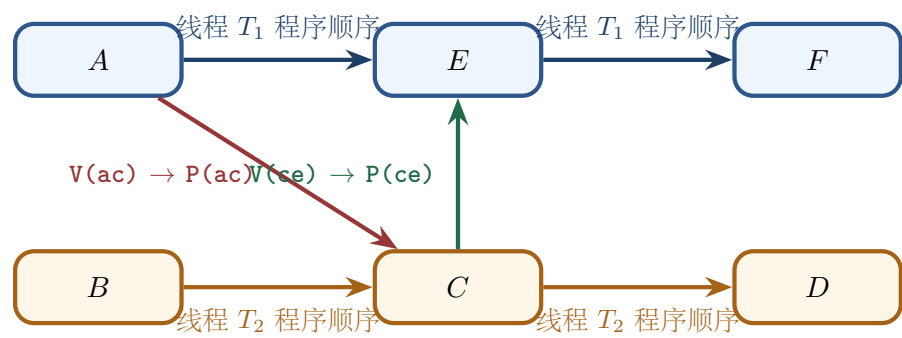
$$A \rightarrow C, B \rightarrow C, C \rightarrow D, C \rightarrow E, E \rightarrow F$$

线程：

$T_1 = \{A, E, F\}, \quad T_2 = \{B, C, D\}.$

只保留跨线程边：

$A \rightarrow C, \quad C \rightarrow E.$



于是：

```
semaphore ac = 0, ce = 0;

T1() {
    A();
    V(ac);
    P(ce);
    E();
    F();
}

T2() {
    B();
    P(ac);
    C();
    V(ce);
    D();
}
```

9.3 新增模式：Barrier 与 fork/join

Fork/Join 线程 A 创建工作线程 B，A 必须等待 B 完成：这就是一次性事件。

$S_{done} = 0, \quad B : V(S_{done}), \quad A : P(S_{done}).$

Barrier 若 N 个线程必须全部完成阶段 1 后才能进入阶段 2，核心是不变量：

$\text{phase2 begins} \Rightarrow \text{count} = N.$

简单一次性 barrier 可用受保护计数器 + gate 信号量构造；可复用 barrier 则需要防止下一轮线程混入上一轮，属于更高阶变式。

10 从 PV 到管程：wait 为什么必须“释放并睡眠”

10.1 问题不是“wait 会不会释放锁”，而是不能丢失唤醒

设线程 T_1 检查：

$count = 0$

准备等待。如果“释放锁”和“真正进入等待队列”之间存在可被其他线程插入的空窗：

```
T1: check count == 0
      <- context switch
T2: count = 1
T2: signal()
T1: sleep
```

那么通知发生时 T_1 还没睡，之后却永久睡下，形成 **lost wakeup**。

因此条件等待必须具有语义：

atomically release mutex and block

被唤醒后，又必须：

reacquire mutex before wait returns

这样线程重新检查共享状态时仍处于受保护环境。

10.2 标准思维模板

```
lock(m);
while (!predicate)
    wait(cv, m);  // 原子释放 m + 阻塞；返回前重新获得 m
use_state();
unlock(m);
```

修改方：

```
lock(m);
change_state();
signal(cv);
unlock(m);
```

两个必须纠正的直觉

- `signal()` 并不等于“条件一定成立且被唤醒者立即运行”；它通常只是使等待者有机会变为可运行状态。
- 条件变量本身不是条件；真正的条件是共享状态上的布尔谓词，所以等待者醒来后必须用 `while` 重检。

10.3 管程是什么

管程可以概括为：

Shared State + Operations + Implicit Mutual Exclusion + Condition Variables

它的核心价值是把“共享状态及其合法同步协议”封装在一个模块中，减少调用者随意破坏不变量的机会。

11 死锁：从等待依赖到资源状态安全性

死锁分析核心控制流

Resource State → Wait Dependency → Deadlock Conditions → State Classification
→ Prevention / Avoidance / Detection → Recovery → Cost

面对任何死锁题目，不再从盲目记忆算法步骤开始，而是固定问 5 个核心问题：

- 1. 谁持有什么？（Allocation）
- 2. 谁还在等什么？（Request/Need）
- 3. 这个等待有没有形成不可消解的依赖？（Wait Dependency / Cycle）
- 4. 题目是在问 Prevention、Avoidance 还是 Detection？
- 5. 我现在掌握的是未来最大需求 Need，还是当前未满足请求 Request？

11.1 死锁到底是什么

死锁 (Deadlock) 的本质是并发执行流之间形成了不可消解的资源等待闭环。多个执行流因相互等待对方持有的不可剥夺资源而全部处于阻塞状态，导致整个系统无法向前推进。

死锁不等于长时间等待 (Long wait)。长时间等待是暂时的，一旦持有资源的线程释放锁即可唤醒；而死锁的等待链形成了环且无法自动解除，属于永久性阻塞。

死锁推理的前置假设：资源类型区分

分析死锁时，不能默认系统资源都是同一种模型，必须区分：

- **Reusable Resource (可重用资源)**：如 CPU、内存页、物理设备、锁/Mutex。一次只能被一个进程使用，使用完毕后被释放重用，总数保持不变。经典银行家算法与死锁检测建立在此模型上。
- **Consumable Resource (可消耗资源)**：如消息 (Message)、信号 (Signal)、事件许可 (Event Token)。由生产者创建，被消费者接收后即被消费消耗，数量动态变化。

异常并发现象	本质驱动原因与系统表现
死锁 (Deadlock)	多个执行流因相互等待对方持有的许可而全部阻塞，系统整体与个体均无法推进
活锁 (Livelock)	执行流未阻塞且持续改变状态/重试，但由于缺乏协调（如持续对称退让）导致整体无法推进
饥饿 (Starvation)	系统整体在推进（无死锁），但特定执行流因调度策略不公长期拿不到资源

11.2 等待图与资源分配图

分析死锁依赖时，不能仅看代码表面，必须区分两类依赖图：

- 1. **Wait-for Graph (等待图)**：节点仅包含进程 P_i 。边 $P_i \rightarrow P_j$ 表示 P_i 正在等待只能由 P_j 释放的资源。
- 2. **Resource-Allocation Graph (资源分配图)**：节点包含进程 P_i 与资源类 R_j 。请求边 $P_i \rightarrow R_j$ 表示 P_i 正在申请资源 R_j ；分配边 $R_j \rightarrow P_i$ 表示 R_j 已被分配给 P_i 。

在不同资源实例模型下，图中是否存在“环 (Cycle)”的判定标准截然不同：

- **单实例资源模型（每类资源仅 1 个实例）**：等待图或分配图中的环是死锁的充要条件：

Single-Instance: Cycle $\iff$ Deadlock

- **多实例资源模型（每类资源有多个实例）**：环仅表示存在死锁风险（必要不充分条件）。环内进程可能等待环外持有同类资源且即将结束的进程释放资源，从而打破环。

因此，考场上必须牢记第一处判定边界：

Cycle $\neq$ Universal Deadlock Test

11.3 Coffman 四个必要条件

死锁形成必须同时满足 Coffman 四个必要条件：

Deadlock $\implies$ ME $\wedge$ HW $\wedge$ NP $\wedge$ CW

- 1. **互斥 (Mutual Exclusion, ME)**：资源一次只能被一个进程独占使用。
- 2. **请求并保持 (Hold and Wait, HW)**：进程至少持有一份资源，又提出了新资源请求，同时不释放已有资源。
- 3. **不可剥夺 (No Preemption, NP)**：资源只能由持有进程自愿释放，系统不能强行剥夺。
- 4. **循环等待 (Circular Wait, CW)**：存在进程等待链 $\{P_0, P_1, \dots, P_n\}$ ，使得 P_i 等待 P_{i+1} 持有的资源。

学习这四个条件不要死记硬背名字，而要回答：**为什么缺了一条件，依赖环就无法闭合？**

例如对于不可剥夺 (NP)：如果资源可以被系统安全地剥夺并恢复现场，当 P_1 请求 R_2 受阻时，OS 可以剥夺 P_1 已持有的 R_1 并分配给其他进程，依赖链就被打破了！

需要特别注意准确的边界表述：

仅竞争可安全抢占的资源 $\implies$ 不会因这些资源形成经典资源死锁

不能绝对化地写成“系统只要出现可抢占资源就不会死锁”，因为进程完全可能同时持有着其他不可抢占的资源。

11.4 死锁处理的四种策略层级

死锁处理策略	核心逻辑与适用场景
忽略 (Ostrich / 鸵鸟策略)	假装死锁不发生。适用于死锁发生概率极低、预防/检测开销过高时（如通用桌面 OS）
预防 (Prevention)	在系统设计期破坏 Coffman 四条件之一，静态保证系统绝不发生死锁
规避 (Avoidance)	在运行时动态评估资源请求，若分配后系统处于安全状态才批准（如 Banker's Algorithm）
检测与恢复 (Detection & Recovery)	允许死锁发生，后台定期运行检测算法，发现死锁后通过剥夺/撤销进程恢复

需要澄清一个工程误区：通用操作系统不运行全局 Banker 算法，不等于完全不做死锁工程防范：

No universal Banker $\neq$ No deadlock engineering

例如 Linux 内核广泛采用固定的锁申请顺序 (Lock Ordering)，并使用 `lockdep` 工具在运行时对锁依赖环进行静态/动态检查。

11.5 Prevention：破坏必要条件的机制与代价

破坏的条件	典型实现机制	核心工程代价
Mutual Exclusion	将独占资源虚拟化/共享化（如 SPOOLing）	许多物理资源本质上无法共享
Hold and Wait	静态一次性申请全部资源；或申请新资源前释放已有资源	资源利用率极低、等待时间显著增加
No Preemption	申请受阻时释放已持有的资源，稍后重新申请	仅适用于易保存恢复现场的资源（内存/寄存器），有 Rollback 开销
Circular Wait	给资源建立全局唯一编号，强制按递增顺序申请	降低编程灵活性，增加模块间耦合

SPOOLing 概念精确说明

SPOOLing 并没有把物理打印机本身变成共享资源。物理打印机依旧是独占设备；SPOOLing 是通过守护进程 (Spooler Daemon) 集中接管物理设备，使多个用户进程可以并发提交逻辑打印任务。

在真实工程中，**全序资源申请 (Global Resource Ordering)** 是最广泛采用的死锁预防思想（例如 Linux 目录锁机制规定必须按 inode 地址或层级顺序加锁）。

11.6 State Classification：Safe / Unsafe / Deadlocked 集合关系

系统的资源分配状态可划分为三个包含关系的集合：

$$\text{Deadlocked} \subset \text{Unsafe} \subset \text{All Allocation States}$$

即：

$$\text{Deadlocked} \implies \text{Unsafe} \quad \text{但} \quad \text{Unsafe} \not\Rightarrow \text{Deadlocked}$$

Unsafe 状态的本质含义

Unsafe State 不等于此刻系统已经陷入死锁！Unsafe 的真实含义是：从当前状态出发，系统无法保证无论未来进程如何提出请求，都一定存在一条能让所有进程顺利完成的安全序列。

如果未来进程实际提出的资源请求少于其声明的最大需求 Need，处于 Unsafe 状态的系统完全可能安全运行完毕而不发生死锁。

11.7 Avoidance 与银行家算法 (Banker's Algorithm)

银行家算法的本质是状态转换试探 (Tentative Transition)。

核心数据结构 设系统有 n 个进程， m 类资源：

- Available[j]：资源 R_j 当前未被分配的可用实例数。
- Max[i, j]：进程 P_i 对资源 R_j 的最大需求数。
- Allocation[i, j]：进程 P_i 当前已分配到的资源 R_j 实例数。
- Need[i, j] = Max[i, j] - Allocation[i, j]：进程 P_i 未来还需要资源 R_j 的最大数量。

资源请求 Request_i 的三关校验 当进程 P_i 发出资源请求 Request_i 时，必须依次通过三关：

1. 第一关：合法性校验

$$\text{Request}_i \leq \text{Need}_i$$

若违反，说明请求超过了其声明的最大需求，抛出非法请求错误。

2. 第二关：充沛性校验

$$\text{Request}_i \leq \text{Available}$$

若违反，说明“当前可用资源不足 (**Insufficient Now**)”， P_i 必须阻塞等待。注意：此时不是 Unsafe，只是暂时无法满足。

3. 第三关：试探分配与安全性测试

系统假设批准请求，试探性更新状态：

$$\text{Available}' = \text{Available} - \text{Request}_i$$

$$\text{Allocation}_i' = \text{Allocation}_i + \text{Request}_i$$

$$\text{Need}_i' = \text{Need}_i - \text{Request}_i$$

随后调用安全性算法 **SafetyTest(State')**：

- 若 State' 安全 $\implies$ **Commit** (正式批准分配)；
- 若 State' 不安全 $\implies$ **Rollback** (撤销试探分配， P_i 恢复阻塞等待)。

Tentative Transition → Safety Check → Commit / Rollback

11.8 安全算法 (Safety Algorithm) 极简理解

安全算法通过模拟“进程按某种顺序依次获得所需全部资源并运行结束归还资源”来寻找安全序列：

- 1. 初始化：Work = Available, Finish[i] = false (i = 1, ..., n)。
- 2. 寻找满足条件的进程 P_i:

$$\text{Finish}[i] == \text{false} \quad \wedge \quad \text{Need}_i \leq \text{Work}$$

- 3. 若找到，模拟 P_i 拿到 Need_i 后运行完成，归还其持有的全部资源：

$$\text{Work} \leftarrow \text{Work} + \text{Allocation}_i, \quad \text{Finish}[i] = \text{true}$$

返回步骤 2。

- 4. 若所有进程 Finish[i] == true，则系统处于安全状态；若最终仍有进程 Finish[i] == false，则处于不安全状态。

为什么 Work 递增的是 Allocation 而不是 Max?

学生最容易混淆为什么 $\text{Work}' = \text{Work} + \text{Allocation}_i$ 。
因为 P_i 运行需要从 Work 中拿走它还缺少的 Need_i。P_i 执行完成后归还的是它拿走的 Need_i 以及原先已持有的 Allocation_i（共归还 Need_i + Allocation_i）。
系统之前已经扣除了 Need_i，因此归还后系统的净资源增加量恰好就是 P_i 原先占有的部分：

$$\text{Work}' = (\text{Work} - \text{Need}_i) + (\text{Need}_i + \text{Allocation}_i) = \text{Work} + \text{Allocation}_i$$

11.9 Avoidance vs Detection：考场核心辨析

死锁规避 (Banker) 与死锁检测 (Detection) 在算法结构上相似，但语义与触发时机有本质不同。考场上必须对比澄清：

对比维度	死锁规避 (Avoidance / Banker)	死锁检测 (Deadlock Detection)
触发时机	资源分配之前（事前试探）	允许分配之后（事后定期/阻塞时检测）
核心目的	保证系统绝不进入 Unsafe 状态	判断系统当前是否已经陷入死锁
关键输入矩阵	Need（未来最大需求）	Request（当前实际正在等待的请求）
矩阵语义	进程未来最多还要再申请多少	进程此刻卡住正在等多少
推演成功含义	存在安全序列，请求可批准	可消去所有进程，当前没有死锁
推演失败含义	Unsafe，拒绝当前分配试探	剩余无法消去的进程处于死锁状态
一句话总结	Banker: Can everybody still finish?	Detection: Who is already unable to finish?

11.10 Deadlock Detection（死锁检测算法）

死锁检测算法不预测未来最大需求 Need，只关注此刻卡住的实际请求 Request_i 能否被满足。

多实例资源检测（消去法）

1. 初始化：Work = Available。对于 $\text{Allocation}_i \neq 0$ 的进程， $\text{Finish}[i] = \text{false}$ ；若 $\text{Allocation}_i == 0$ ，则 $\text{Finish}[i] = \text{true}$ 。

2. 寻找进程 P_i 满足：

$$\text{Finish}[i] == \text{false} \quad \wedge \quad \text{Request}_i \leq \text{Work}$$

3. 若找到，说明 P_i 当前请求可被满足，模拟其获得资源运行结束并释放已占有的资源：

$$\text{Work} \leftarrow \text{Work} + \text{Allocation}_i, \quad \text{Finish}[i] = \text{true}$$

重复步骤 2。

4. 算法终止时，若存在 $\text{Finish}[i] == \text{false}$ 的进程，则这些进程处于死锁状态。

消去法之所以不需要 Max 矩阵，就是因为它只问“现在卡住的请求能否解决”，一旦解决即可假定其能归还资源。

11.11 Recovery：死锁恢复与代价评估

当检测算法确认存在死锁后，系统必须执行恢复。恢复途径分为进程终止与资源剥夺：

1. 进程终止 (Process Termination)：

- 一次性终止所有死锁进程（简单但代价极大）；
- 逐个终止进程，每终止一个重新运行检测算法，直到死锁环消除。

2. 资源抢占 (Resource Preemption)：

- 涉及三个核心决策：选择牺牲者 (Victim selection)、回滚 (Rollback) 与避免饥饿 (Prevent starvation)。

恢复决策统一遵循代价最小化函数：

$$\text{Recovery Cost} = \text{LostWork} + \text{RollbackCost} + \text{ResourceCost} + \text{StarvationRisk}$$

选择牺牲者进程 (Victim) 时综合评估 6 个维度：进程优先级、已运行时间与已完成工作量、尚需多少时间完成、当前持有多少资源、终止可释放多少资源、已被回滚/剥夺的次数。

11.12 考场判题协议 (Exam Protocol)

先判问题类型，再选算法

- 在 408 考场上拿到死锁相关题目，按以下逻辑协议快速定位：
- 问“死锁形成的必要条件是什么？” $\implies$ Coffman 四条件 (ME, HW, NP, CW)
 - 问“当前分配状态是否安全？” $\implies$ 运行 Safety Algorithm (看 Need 矩阵)
 - 问“这个资源请求是否应被批准？” $\implies$ Resource-Request 三关 + Safety Algorithm
 - 问“系统现在是否已经陷入死锁？” $\implies$ 运行 Deadlock Detection 消去法 (看 Request 矩阵)
 - 问“资源分配图里有环就一定是死锁吗？” $\implies$ 先看每类资源实例数 (单实例 死锁，多实例 充要)
 - 问“如何在设计/运行期保证死锁绝不发生？” $\implies$ Prevention (破坏 4 条件) / Avoidance (Banker)
 - 问“死锁已经发生怎么处理？” $\implies$ Recovery (终止进程 / 抢占资源)

12 经典 PV 题不要背答案：把它们看成“并发结构模式库”

经典同步题型	真正抽象结构	解题第一反应与建模板块
生产者-消费者	容量计数 + 共享结构互斥	写 $0 \leq count \leq N$, 寻找 empty / full / mutex
读者-写者	群体准入 + 元数据保护 + 公平策略	谁可并行？谁必须独占？谁记录群体状态？
哲学家进餐	多资源同时需求 + 循环等待	画资源依赖图；考虑全序、准入限制或原子资格判断
吸烟者问题	多路条件分派 / Selective Wakeup	不同条件是否应有不同事件信号量？
理发师问题	有限准入 + 服务/请求双向会合	分清等待容量、请求数量、服务准备三类状态
前驱图 (DAG)	Happens-Before 显式前驱边	只为跨执行流依赖建立初值为 0 的事件信号量
Barrier 屏障	阶段协同同步	所有人到齐之前无人能越过阶段边界
Fork / Join	一次性异步完成事件	完成者执行 V，等待者执行 P，初值设为 0

13 408 大题作答模板：从自然语言到 PV 伪代码

13.1 第一遍：只翻译业务，不写 P/V

先写：

```
producer: produce -> put
consumer: get -> consume
```

或者：

```
reader: enter_read -> read -> leave_read
writer: enter_write -> write -> leave_write
```

避免一边读题一边随手插 P/V，导致局部正确、整体矛盾。

13.2 第二遍：建立状态表

建议草稿固定写：

对象 / 资源	初始状态与许可数量	谁会改变/消费该许可?
共享数据实体	例如 <code>buffer / count</code>	生产者 / 消费者（通过临界区保护）
互斥许可	<code>Initial = 1</code>	进入/离开临界区的执行流成对 P/V
资源许可	<code>Initial = N</code> 或 0	生产方 V 增加，消费方 P 减少
事件许可	<code>Initial = 0</code>	前驱动作完成者 V，后继等待者 P

13.3 第三遍：在“等待点”和“制造条件点”成对放置 P/V

每一个 P 都应能回答：

我现在具体缺少哪一份事实/资源/许可？谁能够在未来执行 V 制造它？

每一个 V 都应能回答：

我刚刚完成了什么状态变化，使哪类等待者现在可能继续？

13.4 第四遍：四类证明

- 1. **Safety**：临界区是否互斥？容量是否越界？前驱是否被违反？
- 2. **Deadlock**：有没有“持有 A 等 B，而制造 B 的人必须先拿 A”？有没有资源环？
- 3. **Liveness**：每个阻塞点未来是否存在可达的 V？
- 4. **Performance**：耗时业务是否被不必要地放进互斥区？是否人为串行化？

14 高频错因诊断表

错误现象	根本认知缺口	修复与排查提问
<code>mutex</code> 初值乱设	没理解“一个互斥进入许可”	系统初始能允许几人同时进入？
<code>full/empty</code> 写反	没从初始物理状态推许可	一开始已有多少产品？多少空位？
看到共享变量就全加锁	没区分共享状态与不变量	哪些多步更新真的不能交错？
锁包住生产/计算耗时操作	不理解临界区粒度	哪部分只是私有计算？

错误现象	根本认知缺口	修复与排查提问
先拿 <code>mutex</code> 后等待容量死锁	没画等待依赖	制造我等待条件的人是否也需要我持有的锁?
读者写者漏保护 <code>count</code>	忽视元数据也是共享状态	谁在决定“第一个/最后一个”?
<code>V</code> 了错误信号量	没把 <code>V</code> 当成“制造事实”	我刚刚制造了哪一种可消费条件?
前驱图边全设信号量	没区分线程内部程序顺序	这条依赖是否已经由同一线程顺序保证?
哲学家只背左右筷顺序	没看到资源等待环	是否存在全序或原子获取策略?
把唤醒当作立即运行	混淆 <code>Blocked→Ready</code> 与 <code>Ready→Running</code>	谁还必须经过 <code>scheduler</code> ?
把 <code>semaphore</code> 当 <code>mutex</code>	混淆许可与所有权	这里管理的是“谁拥有锁”还是“还有几份许可”?

15 考场速查：PV 十问

拿到 PV 大题，按顺序问

1. 有几类执行流?

2. 它们共享哪些状态/资源?

3. 每个共享状态的初值是什么?

4. 哪些状态更新必须互斥?

5. 哪些动作必须等待某个条件/前驱?

6. 每个等待条件在现实中对应什么“许可”? 初值多少?

7. 谁在什么动作完成后制造这个许可 (V)?

8. 是否持锁等待了只能由竞争者制造的条件?

9. 是否存在循环等待、饥饿或无意义的大临界区?

10. 最终伪代码是否保持所有不变量，并且每个阻塞点都有未来可达的唤醒路径?

16 专题总结：把 PV 从代码模板提升为状态约束与语义建模

最终核心模型

Flows → Shared State → Interleavings → Invariant → Constraints

→ Semaphore/PV → Progress → Cost

不要看到生产者-消费者就机械写 `empty/full/mutex`。按以下顺序建模：

1. 先找到谁在等待什么事实；

2. 再找到**谁能够制造这个事实**；
3. 用信号量把事实变成可被原子消费/生产的许可；
4. 用互斥保护共享状态的不变量；
5. 最后证明所有允许交错下系统既不会进入坏状态，也不会因为等待环而无法推进。

与进程调度的接口

P 操作可能使执行流从 Running 进入 Blocked，V 操作可能使等待者从 Blocked 进入 Ready；Ready→Running 仍由调度器决定。PV 只表达许可和等待，不直接分配 CPU。

边界检查：四句不能绝对化

四句不应绝对化的话

1. “所有 PV 题都必须先同步 P 后互斥 P”——标准有界缓冲区正确，但更一般原则是避免持锁等待依赖环。
2. “V 的顺序永远不重要”——简单模板里可能不影响 Safety，但会影响唤醒竞争、性能，复杂协议还可能影响语义。
3. “二元信号量就是 mutex”——考试可用于互斥，但严格语义中 mutex 有所有权，semaphore 表示许可。
4. “某个哲学家解法一定无饥饿”——无死锁不自动等于公平；饥饿还取决于调度和唤醒策略。